

Technical Assignment

Position: Back-End Developer (PHP / Laravel)

Introduction

Thank you for taking the time to participate in our hiring process.

The purpose of this assignment is not to evaluate your ability to build simple CRUD operations. Instead, we want to assess how you approach real-world backend problems, make technical decisions, write maintainable code, and test your implementation.

The assignment is designed to take approximately **6–8 hours**. You have **2 days** to complete and submit it.

If you are unable to finish every part, prioritize the core requirements and explain any incomplete sections in your README.

AI Tools

You are welcome to use AI tools such as ChatGPT, Claude, GitHub Copilot, or similar assistants.

However, we expect that you:

- Review all generated code carefully.
- Fully understand your implementation.
- Can explain and defend your technical decisions during the interview.

Our goal is to evaluate your engineering skills—not whether you used AI.

Scenario

You are building part of an online platform that sells **Apple IDs** and **application subscriptions**.

The system should:

- Accept customer orders.
- Manage products with limited inventory.
- Confirm payments through a payment gateway webhook.
- Deliver purchased accounts asynchronously after payment.

Assume the system is running under production traffic where multiple users may attempt to purchase the last available item simultaneously.

Core Requirements

1. Safe Checkout (Concurrency)

Implement a checkout process that prevents overselling.

Use an appropriate database-level concurrency strategy (Pessimistic Locking, Optimistic Locking, or an equivalent approach).

Write a test demonstrating that concurrent requests cannot oversell the inventory.

2. Idempotency

The order creation endpoint must support an **Idempotency-Key** header.

Submitting the same request with the same key must:

- Not create a duplicate order.
 - Return the original response.
-

3. Payment Webhook

Create a webhook endpoint that:

- Validates an HMAC signature.
 - Rejects invalid requests with HTTP 401.
 - Enqueues the account delivery job after successful payment confirmation.
-

4. Queue & Asynchronous Delivery

Account delivery should be processed by a background job.

Simulate an external service with:

- Artificial latency.
- Approximately a 30% random failure rate.

If delivery fails:

- Retry up to three times.
- Use exponential backoff.
- Mark the order as Failed if all retries are exhausted.

5. Authorization

Implement authorization so that:

- Customers can access only their own orders.
- Administrators can view, cancel, or refund any order.

Use Laravel Policies or Gates instead of manual authorization checks inside controllers.

Optional Features

If time permits, you may also implement:

- Rate limiting
 - Redis caching
 - Structured logging
 - OpenAPI documentation or a Postman Collection
-

Written Questions

Answer the following questions in your README (or a separate markdown file):

1. How would you scale this system to support 100,000 orders per day? What would become the first bottleneck?
2. What do you consider the biggest security risk in this system, and how would you mitigate it?

A concise answer (around half a page per question) is sufficient.

Minimum Tests

Include tests covering:

- Successful checkout
- Oversell prevention
- Idempotency

- Webhook validation
 - Authorization
-

Priorities

If you run out of time, prioritize the following:

1. Oversell prevention
 2. Queue & Retry
 3. Idempotency
 4. Webhook
 5. Authorization
 6. Optional features
-

Deliverables

Please submit:

- A Git repository or ZIP archive
 - A README including:
 - Setup instructions
 - Assumptions
 - Any incomplete features
 - Your written answers
-

Evaluation Criteria

Your submission will be evaluated based on:

- Correctness
- Code quality
- Architecture
- Testing
- Error handling
- Security considerations
- Documentation
- Technical decision-making

Good luck!